INTERVIEW PREP SERIES

Node.js

Interview Questions & Easy Answers

---

52 questions · 20 marked ★ most asked

Covers the Node runtime, event loop, modules, npm, streams and buffers, file system, async patterns, error handling, clustering, security and authentication basics, as asked in backend and MERN rounds.

How to use this document

- Questions go from basic to advanced inside each section. Freshers should master the ★ ones first.
- Answers are written in simple words, the way you would say them in an interview. Learn the idea, not the exact sentences.
- Questions are based on technical rounds reported by candidates on GeeksforGeeks, LeetCode Discuss, Glassdoor, Naukri and similar interview-experience pages, plus the standard question banks interviewers draw from.
- Read one section a day and say each answer out loud once. Speaking it is what makes it stick.

Basics

Q1. What is Node.js? ★ Most asked

Node.js is a JavaScript runtime built on Chrome's **V8 engine** that lets you run JavaScript outside the browser, mainly on servers. It is single-threaded, event-driven and uses non-blocking I/O, which makes it great for APIs, real-time apps (chat) and handling many connections at once.

Q2. Is Node.js single-threaded? How does it handle many requests? ★ Most asked

Your JavaScript code runs on a single main thread. But slow work like file reading, database calls and network requests is handed off to the system or to **libuv's thread pool**. While that work happens, Node keeps accepting new requests. When the work finishes, its callback is put in a queue and the **event loop** runs it. So one thread can serve thousands of users, as long as you don't block it with heavy CPU work.

Q3. What is the event loop in Node.js? ★ Most asked

The event loop is the mechanism that lets Node do non-blocking I/O on a single thread. It continuously checks for completed tasks and runs their callbacks in phases:

- **Timers**: `setTimeout`, `setInterval` callbacks.
- **Pending callbacks**: some system I/O callbacks.
- **Poll**: gets new I/O events and runs their callbacks.
- **Check**: `setImmediate` callbacks.
- **Close callbacks**: e.g. `socket.on('close')`.

Between each phase, Node runs microtasks: `process.nextTick` first, then resolved promises.

Q4. What is the difference between blocking and non-blocking code? ★ Most asked

Blocking code stops everything until it finishes, e.g. `fs.readFileSync`. Non-blocking code starts the task and moves on, and a callback/promise handles the result later, e.g. `fs.readFile` or `fs.promises.readFile`. In a server you should use non-blocking code so one slow request doesn't freeze all users.

Q5. What is libuv?

libuv is a C library used by Node that provides the event loop, the thread pool (default 4 threads) and asynchronous handling of files, DNS, networking and timers across operating systems.

Q6. What is V8?

V8 is Google's open-source JavaScript engine (used in Chrome). It compiles JavaScript into machine code using JIT (just-in-time) compilation, which makes Node fast.

Q7. Where is Node.js a bad choice?

For CPU-heavy work like video encoding, big image processing or heavy calculations, because they block the single thread. You can use worker threads, or another language for such services.

Q8. What is the difference between Node.js and a browser's JavaScript?

Both use JavaScript, but the browser has the DOM, `window` and `document`, while Node has no DOM and instead provides modules for files (`fs`), servers (`http`), OS info and processes. The global object is `window` in the browser and `global`/`globalThis` in Node.

Modules and npm

Q9. What are modules in Node.js? What are the types? ★ Most asked

A module is a reusable file of code. Types:

- **Core modules**: built in, e.g. `fs`, `http`, `path`, `os`, `events`, `crypto`.
- **Local modules**: files you create.
- **Third-party modules**: installed from npm, e.g. `express`, `mongoose`.

Q10. What is the difference between `require` and `import`? ★ Most asked

`require` is the CommonJS syntax (`module.exports`), loads modules synchronously and can be called anywhere. `import`/`export` is the ES Module syntax, is static (top of the file), supports tree-shaking and is the modern standard. To use ESM in Node, set `"type": "module"` in package.json or use `.mjs` files.

Q11. What is the difference between `module.exports` and `exports`?

`exports` is just a shortcut reference to `module.exports`. You can add properties to it (`exports.add = ...`), but if you assign a new value to `exports` directly, the link breaks. What actually gets exported is always `module.exports`.

Q12. What is npm and what is package.json? ★ Most asked

npm (Node Package Manager) installs and manages third-party packages. `package.json` describes your project: name, version, scripts (`npm start`, `npm run dev`), dependencies and devDependencies.

Q13. What is the difference between dependencies and devDependencies?

`dependencies` are needed to run the app in production (express, mongoose). `devDependencies` are only needed during development (nodemon, jest, eslint). Install a dev one with `npm install -D nodemon`.

Q14. What is package-lock.json?

It records the exact versions of every installed package and sub-package, so every developer and server installs exactly the same versions. You should commit it to Git.

Q15. What is the difference between npm and npx?

`npm` installs packages. `npx` runs a package's command without installing it globally, e.g. `npx create-vite@latest`.

Q16. What does `^` and `~` mean in package versions?

`^1.4.2` allows updates to any 1.x.x version (minor and patch). `~1.4.2` allows only patch updates (1.4.x). No symbol means an exact version.

Core modules

Q17. How do you create a simple HTTP server in Node.js? ★ Most asked

```
const http = require("http");

const server = http.createServer((req, res) => {
  res.writeHead(200, { "Content-Type": "text/plain" });
  res.end("Hello World");
});
```

```
server.listen(3000, () => console.log("Server on port 3000"));
```

Q18. How do you read and write files in Node.js? ★ Most asked

```
const fs = require("fs/promises");

async function demo() {
  await fs.writeFile("note.txt", "Hello");
  const data = await fs.readFile("note.txt", "utf8");
  console.log(data);
}
```

There are also sync versions (`readFileSync`) and callback versions (`fs.readFile(path, cb)`).

Q19. What is the `path` module used for?

To work with file paths safely across operating systems: `path.join(__dirname, "public", "index.html")`, `path.extname("a.png")`, `path.basename()`.

Q20. What are `__dirname` and `__filename`?

`__dirname` is the absolute path of the folder containing the current file, and `__filename` is the absolute path of the file itself. They exist in CommonJS; in ES modules use `import.meta.dirname` (newer Node) or `fileURLToPath(import.meta.url)`.

Q21. What is the EventEmitter? ★ Most asked

The `events` module lets objects emit named events and other code listen to them. Many Node features (streams, HTTP server) are built on it.

```
const EventEmitter = require("events");
const emitter = new EventEmitter();
emitter.on("order", item => console.log("Ordered:", item));
emitter.emit("order", "Pizza");
```

Q22. What is the `process` object?

A global object with information about the running Node process: `process.env` (environment variables), `process.argv` (command-line arguments), `process.exit()`, `process.pid`, and `process.on("uncaughtException")`.

Q23. What is the `os` module?

It gives information about the operating system: `os.cpus()`, `os.totalmem()`, `os.freemem()`, `os.platform()`, `os.hostname()`.

Streams and buffers

Q24. What are streams? What are the types? ★ Most asked

Streams process data piece by piece (in chunks) instead of loading it all into memory, which is ideal for big files and videos. Types:

- **Readable**: you read from it (`fs.createReadStream`).
- **Writable**: you write to it (`fs.createWriteStream`, HTTP response).
- **Duplex**: both read and write (TCP socket).
- **Transform**: duplex that changes data on the way (zlib compression).

```
fs.createReadStream("big.mp4").pipe(res);
```

Q25. What is a Buffer?

A Buffer is a fixed-size chunk of raw binary data in memory. Node uses buffers to handle data from files, networks and streams before it's converted to strings. `Buffer.from("hello")`.

Q26. What is piping?

`readable.pipe(writable)` connects a readable stream to a writable stream so data flows automatically. It also handles backpressure (slowing down when the writer is busy). `stream.pipeline()` is a safer version with error handling.

Async patterns and errors

Q27. What is the difference between `process.nextTick()`, `setImmediate()` and `setTimeout()`? ★ Most asked

- `process.nextTick()` runs right after the current operation, before any other I/O or timers (highest priority).
- Promise callbacks run next (microtasks).
- `setTimeout(fn, 0)` runs in the timers phase.
- `setImmediate()` runs in the check phase, after I/O.

Inside an I/O callback, `setImmediate` always runs before `setTimeout(fn, 0)`.

Q28. What is callback hell and how do you fix it? ★ Most asked

Deeply nested callbacks, each depending on the previous one, that become hard to read and handle errors in. Fix it with promises, `async/await`, `util.promisify`, and by breaking code into small functions.

Q29. How do you handle errors in Node.js? ★ Most asked

- Callbacks: check the first `err` argument (error-first callbacks).
- Promises: `.catch()`.
- async/await: `try/catch`.
- Express: central error-handling middleware.
- Global safety nets: `process.on("unhandledRejection")` and `process.on("uncaughtException")` to log and restart the process.

Q30. What is an error-first callback?

A Node convention where the first argument of a callback is the error and the second is the result.

```
fs.readFile("a.txt", "utf8", (err, data) => {
  if (err) return console.error(err);
  console.log(data);
});
```

Q31. What is `util.promisify`?

It converts a callback-based function into one that returns a promise, so you can use `await` with it: `const readFile = util.promisify(fs.readFile);`.

Scaling and performance

Q32. What is clustering in Node.js? ★ Most asked

Node runs on one CPU core by default. The `cluster` module (or PM2 in cluster mode) starts multiple worker processes, one per core, that share the same port. This uses all CPU cores and improves throughput. If one worker crashes, others continue.

Q33. What are worker threads?

`worker_threads` let you run CPU-heavy JavaScript in parallel threads within the same process, so the main event loop is not blocked. Use them for heavy calculations, not for I/O.

Q34. What is the difference between cluster and worker threads?

Cluster creates separate processes, each with its own memory and event loop, mostly to handle more requests. Worker threads are threads inside one process that can share memory, mostly for CPU-heavy tasks.

Q35. What is PM2?

PM2 is a production process manager for Node. It keeps your app running, restarts it after crashes, runs it in cluster mode, manages logs and can start the app on server reboot.

Q36. How do you improve Node.js performance?

Avoid blocking code, use async I/O, cache with Redis, use clustering/PM2, compress responses (gzip), use streams for large data, add database indexes, and use pagination.

Q37. What is a memory leak in Node and how do you find it?

A memory leak happens when memory is used but never freed, often from global variables, ever-growing caches/arrays, or event listeners that are never removed. Find it by watching memory usage over time and taking heap snapshots with Chrome DevTools (`node --inspect`).

Security, auth and APIs

Q38. What are environment variables and how do you use them? ★ Most asked

Values set outside the code, like DB URLs, API keys and the port, so secrets aren't committed to Git. Store them in a `.env` file (added to `.gitignore`) and load them with `dotenv` (or `node --env-file=.env` in newer Node). Read them with `process.env.MONGO_URI`.

Q39. What is JWT and how does JWT authentication work? ★ Most asked

JWT (JSON Web Token) is a signed token with three parts: header, payload, signature.

1. The user logs in with email/password.

2. The server checks the password and creates a token: `jwt.sign({ id: user._id }, SECRET, { expiresIn: "1d" })`.

3. The client stores the token (preferably in an httpOnly cookie) and sends it with each request.

4. The server verifies it with `jwt.verify()` in middleware before giving access.

It is stateless, so the server does not need to store sessions.

Q40. How do you store passwords securely? ★ Most asked

Never store plain passwords. Hash them with **bcrypt** (which adds a salt automatically), then compare during login.

```
const hash = await bcrypt.hash(password, 10);
const ok = await bcrypt.compare(inputPassword, hash);
```

Q41. What is the difference between authentication and authorization?

Authentication checks **who you are** (login). Authorization checks **what you are allowed to do** (e.g. only admins can delete users).

Q42. What is CORS? ★ Most asked

CORS (Cross-Origin Resource Sharing) is a browser security rule that blocks a page from calling an API on a different origin (domain/port) unless the server allows it. In Express, use the `cors` package: `app.use(cors({ origin: "http://localhost:5173" }))`.

Q43. What are common security practices in Node apps?

Validate and sanitise input, hash passwords, use HTTPS, set secure headers with **helmet**, add rate limiting, keep secrets in env variables, avoid `eval`, update packages (`npm audit`), and use parameterised queries to prevent injection.

Q44. What is REST? What are the HTTP methods? ★ Most asked

REST is a style for designing APIs around resources using standard HTTP methods:

- `GET /users` read, `POST /users` create, `PUT /users/1` replace, `PATCH /users/1` partial update, `DELETE /users/1` remove.

REST APIs are stateless and usually return JSON.

Q45. What are common HTTP status codes?

200 OK, 201 Created, 204 No Content, 301 Moved, 400 Bad Request, 401 Unauthorized (not logged in), 403 Forbidden (no permission), 404 Not Found, 409 Conflict, 429 Too Many Requests, 500 Internal Server Error.

Q46. What are WebSockets and Socket.io?

WebSockets keep a two-way, always-open connection between client and server, so the server can push data instantly. Socket.io is a popular Node library built on WebSockets for chat apps, live notifications and multiplayer games.

Q47. What is nodemon?

A development tool that restarts your Node server automatically whenever you save a file. Newer Node versions also have `node --watch`.

Q48. What is middleware?

A function that runs between receiving a request and sending the response, used for logging, authentication, parsing JSON and error handling. It is a core concept in Express (see the Express document).

More frequently asked

Q49. What is the difference between `fs.readFile` and `fs.createReadStream`?

`readFile` loads the whole file into memory before you can use it, which is fine for small files. `createReadStream` reads it in small chunks, so memory stays low even for multi-GB files, and you can start sending data immediately.

Q50. How do you handle graceful shutdown in Node?

Listen for `SIGTERM`/`SIGINT`, stop accepting new requests with `server.close()`, finish in-flight requests, close the database connection, then `process.exit(0)`. This prevents lost requests during deploys.

Q51. What is the difference between `spawn`, `exec` and `fork`?

All are in `child_process`. `exec` runs a command and buffers the whole output (good for small outputs). `spawn` streams the output (good for large outputs or long processes). `fork` starts a new Node process with a message channel to talk to the parent.

Q52. What is caching and how do you use Redis with Node?

Caching stores frequently requested data in fast memory so the database isn't hit every time. With Redis, check the cache first; if the key is missing, query the DB, save the result in Redis with an expiry (`SET key value EX 60`), and return it.